

Tasks

Part A: Dataset Understanding

Q1.

Load the dataset and display the first five records.

code:

```
import pandas as pd
```

```
df = pd.read_csv("Dataset 2.csv")
```

```
print(df.head())
```

Output:

```
[2]
✓ Os
import pandas as pd
df = pd.read_csv("Dataset 2.csv")
print(df.head())
```

	UserID	Age	Gender	SubscriptionType	WatchHoursPerWeek	DevicesUsed	\
0	1001	22	Female	Basic	23	5	
1	1002	55	Male	Basic	9	4	
2	1003	49	Male	Basic	8	3	
3	1004	39	Female	Premium	19	5	
4	1005	38	Female	Premium	23	5	

	FavoriteGenre	AdClicks	MonthlySpend	SubscriptionRenewed
0	Comedy	13	353	No
1	Drama	14	317	Yes
2	Comedy	16	309	No
3	Drama	45	833	Yes
4	Sci-Fi	24	804	Yes

Q2.

Determine the number of rows and columns in the dataset.

code:

```
print("Rows and Columns:", df.shape)
```

Output:

```
print("Rows and Columns:", df.shape)
```

```
... Rows and Columns: (750, 10)
```

Q3.

Display all column names.

code:

```
print(df.columns)
```

Output:

```
print(df.columns)

... Index(['UserID', 'Age', 'Gender', 'SubscriptionType', 'WatchHoursPerWeek',
          'DevicesUsed', 'FavoriteGenre', 'AdClicks', 'MonthlySpend',
          'SubscriptionRenewed'],
          dtype='object')
```

Q4.

Identify numerical and categorical features.

code:

```
numerical_features = df.select_dtypes(include=['int64', 'float64']).columns
```

```
categorical_features = df.select_dtypes(include=['object']).columns
```

```
print("Numerical Features:")
```

```
print(numerical_features)
```

```
print("Categorical Features:")
```

```
print(categorical_features)
```

Output:

```
numerical_features = df.select_dtypes(include=['int64', 'float64']).columns
categorical_features = df.select_dtypes(include=['object']).columns

print("Numerical Features:")
print(numerical_features)

print("Categorical Features:")
print(categorical_features)

... Numerical Features:
Index(['UserID', 'Age', 'WatchHoursPerWeek', 'DevicesUsed', 'AdClicks',
      'MonthlySpend'],
      dtype='object')
Categorical Features:
Index(['Gender', 'SubscriptionType', 'FavoriteGenre', 'SubscriptionRenewed'], dtype='object')
```

Q5.

Check whether the dataset contains missing values.

code:

```
print(df.isnull().sum())
```

Output:

```
print(df.isnull().sum())
```

```
... UserID          0
   Age             0
   Gender          0
   SubscriptionType 0
   WatchHoursPerWeek 0
   DevicesUsed      0
   FavoriteGenre     0
   AdClicks         0
   MonthlySpend      0
   SubscriptionRenewed 0
   dtype: int64
```

Part B: Exploratory Data Analysis

Q6.

Calculate the average age of users.

code:

```
print("Average Age:", df['Age'].mean())
```

Output:

```
print("Average Age:", df['Age'].mean())
```

```
... Average Age: 41.824
```

Q7.

Determine the average watch hours per week.

code:

```
print("Average Watch Hours:", df['WatchHoursPerWeek'].mean())
```

Output:

```
print("Average Watch Hours:", df['WatchHoursPerWeek'].mean())
```

```
... Average Watch Hours: 14.236
```

Q8.

Find the average monthly spending of users.

code:

```
print("Average Monthly Spending:", df['MonthlySpend'].mean())
```

Output:

```
print("Average Monthly Spending:", df['MonthlySpend'].mean())
```

```
... Average Monthly Spending: 689.9053333333334
```

Q9.

Count the number of users in each subscription category.

code:

```
print(df['SubscriptionType'].value_counts())
```

Output:

```
print(df['SubscriptionType'].value_counts())
```

```
... SubscriptionType
Basic      342
Premium    279
VIP        129
Name: count, dtype: int64
```

Q10.

Determine the percentage of users who renewed their subscriptions.

code:

```
renewed_percentage = df['SubscriptionRenewed'].value_counts(normalize=True) * 100
```

```
print(renewed_percentage)
```

Output:

```
renewed_percentage = df['SubscriptionRenewed'].value_counts(normalize=True) * 100
```

```
print(renewed_percentage)
```

```
... SubscriptionRenewed
No      53.733333
Yes     46.266667
Name: proportion, dtype: float64
```

Part C: Data Preparation

Q11.

Convert categorical features into numerical form.

code:

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
categorical_columns = [
```

```
    'Gender',
```

```
    'SubscriptionType',
```

```
    'FavoriteGenre',
```

```
    'SubscriptionRenewed'
```

```
]
```

```
for col in categorical_columns:
```

```
    df[col] = le.fit_transform(df[col])
```

```
print(df.head())
```

Output:

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

categorical_columns = [
    'Gender',
    'SubscriptionType',
    'FavoriteGenre',
    'SubscriptionRenewed'
]

for col in categorical_columns:
    df[col] = le.fit_transform(df[col])

print(df.head())
```

```
...   UserID  Age  Gender  SubscriptionType  WatchHoursPerWeek  DevicesUsed  \
0      1001   22      0              0              23           5
1      1002   55      1              0              9           4
2      1003   49      1              0              8           3
3      1004   39      0              1             19           5
4      1005   38      0              1             23           5

   FavoriteGenre  AdClicks  MonthlySpend  SubscriptionRenewed
0              1         13           353              0
1              2         14           317              1
2              1         16           309              0
3              2         45           833              1
4              5         24           804              1
```

Q12.

Define the feature set (X) and target variable (y) for subscription renewal prediction.

code:

```
X = df.drop(['SubscriptionRenewed', 'UserID'], axis=1)
```

```
y = df['SubscriptionRenewed']
```

Q13.

Split the dataset into training and testing sets.

code:

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Part D: Decision Tree Classification

Q14.

Train a Decision Tree model to predict whether a user will renew their subscription.

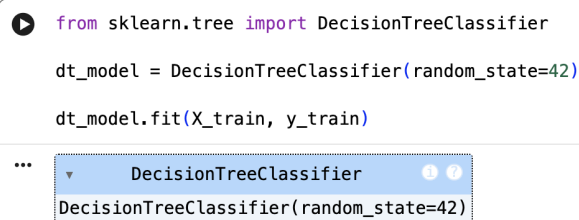
code:

```
from sklearn.tree import DecisionTreeClassifier
```

```
dt_model = DecisionTreeClassifier(random_state=42)
```

```
dt_model.fit(X_train, y_train)
```

Output:

A screenshot of a Jupyter Notebook cell showing the execution of code to train a Decision Tree Classifier. The code includes the import of DecisionTreeClassifier, the creation of the model with random_state=42, and the fitting of the model to training data. The output shows the instantiated DecisionTreeClassifier object.

```
from sklearn.tree import DecisionTreeClassifier

dt_model = DecisionTreeClassifier(random_state=42)

dt_model.fit(X_train, y_train)
```

... DecisionTreeClassifier
DecisionTreeClassifier(random_state=42)

Q15.

Evaluate the model using accuracy.

code:

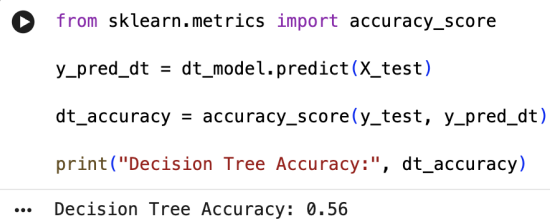
```
from sklearn.metrics import accuracy_score
```

```
y_pred_dt = dt_model.predict(X_test)
```

```
dt_accuracy = accuracy_score(y_test, y_pred_dt)
```

```
print("Decision Tree Accuracy:", dt_accuracy)
```

Output:

A screenshot of a Jupyter Notebook cell showing the execution of code to evaluate the model's accuracy. The code includes the import of accuracy_score, the prediction on test data, the calculation of accuracy, and the printing of the result. The output shows the calculated accuracy of 0.56.

```
from sklearn.metrics import accuracy_score

y_pred_dt = dt_model.predict(X_test)

dt_accuracy = accuracy_score(y_test, y_pred_dt)

print("Decision Tree Accuracy:", dt_accuracy)
```

... Decision Tree Accuracy: 0.56

Q16.

Generate and interpret the confusion matrix.

code:

```
from sklearn.metrics import confusion_matrix
```

```
cm = confusion_matrix(y_test, y_pred_dt)
```

```
print(cm)
```

Output:

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred_dt)
print(cm)
... [[48 34]
     [32 36]]
```

Part E: K-Nearest Neighbors (KNN)

Q17.

Train a KNN classifier with $K = 5$.

code:

```
from sklearn.neighbors import KNeighborsClassifier
```

```
knn_model = KNeighborsClassifier(n_neighbors=5)
```

```
knn_model.fit(X_train, y_train)
```

```
y_pred_knn = knn_model.predict(X_test)
```

Q18.

Compare the accuracy of KNN with the Decision Tree model.

code:

```
knn_accuracy = accuracy_score(y_test, y_pred_knn)
```

```
print("KNN Accuracy:", knn_accuracy)
```

```
print("Decision Tree Accuracy:", dt_accuracy)
```

```
if knn_accuracy > dt_accuracy:
```

```
    print("KNN performed better.")
```

```
else:
```

```
    print("Decision Tree performed better.")
```

Output:

```
▶ knn_accuracy = accuracy_score(y_test, y_pred_knn)

print("KNN Accuracy:", knn_accuracy)
print("Decision Tree Accuracy:", dt_accuracy)

if knn_accuracy > dt_accuracy:
    print("KNN performed better.")
else:
    print("Decision Tree performed better.")

... KNN Accuracy: 0.6333333333333333
    Decision Tree Accuracy: 0.56
    KNN performed better.
```

Part F: Linear Regression

Q19.

Train a Linear Regression model to predict monthly spending.

code:

```
from sklearn.linear_model import LinearRegression
```

```
X_reg = df.drop(['MonthlySpend', 'UserID'], axis=1)
```

```
y_reg = df['MonthlySpend']
```

```
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg,
    y_reg,
    test_size=0.2,
    random_state=42
)
```

```
lr_model = LinearRegression()
```

```
lr_model.fit(X_train_reg, y_train_reg)
```


Output:

```
from sklearn.linear_model import LinearRegression

X_reg = df.drop(['MonthlySpend', 'UserID'], axis=1)

y_reg = df['MonthlySpend']

X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg,
    y_reg,
    test_size=0.2,
    random_state=42
)

lr_model = LinearRegression()

lr_model.fit(X_train_reg, y_train_reg)
```

```
...
  ▾ LinearRegression ⓘ ?
    LinearRegression()
```

Q20.

Predict the monthly spending for a new user and interpret the result.

code:

```
new_user = [[25, 1, 2, 15, 3, 1, 10, 1]]
```

```
predicted_spend = lr_model.predict(new_user)
```

```
print("Predicted Monthly Spending:", predicted_spend[0])
```

Output:

```
new_user = [[25, 1, 2, 15, 3, 1, 10, 1]]

predicted_spend = lr_model.predict(new_user)

print("Predicted Monthly Spending:", predicted_spend[0])

... Predicted Monthly Spending: 1317.8351945433988
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression was fi
warnings.warn(
```

Business Reflection Questions

1. Which factors appear to influence subscription renewal the most?

Factors such as:

- Watch hours per week
- Subscription type
- Monthly spending
- Number of devices used
- Ad clicks

appear to influence subscription renewal the most because they represent user engagement and platform usage.

2. Why is subscription renewal a classification problem?

Subscription renewal is a classification problem because the output variable has categories:

- Yes
- No

The model predicts which class the user belongs to.

3. Why is monthly spending a regression problem?

Monthly spending is a regression problem because the target variable is a continuous numerical value. Regression algorithms are used to predict continuous outputs.

4. Which algorithm performed better for renewal prediction?

The algorithm with the higher accuracy score performed better. Which was KNN.

5. How could the platform use these predictions to improve customer retention?

Netflix can:

- Identify users likely to cancel subscriptions
- Offer personalized recommendations
- Provide discounts or special offers
- Increase engagement through targeted campaigns
- Improve customer satisfaction using viewing behaviour analysis

These strategies can help reduce churn and improve customer retention